**Licence Informatique**

**Parcours MIAGE**

SAE 4 - Réaliser une application informatique complète

Professeurs: Yu LI && Frédéric Fürst

Rapport de Réalisation

Livré le 29/03/2026

Yassir ELMOUDNI
Henry Andrew MEDINA CONDO

Semestre 5

"L'étudiant déclare avoir réalisé ce travail conformément au règlement de l'Université Picardie Jules Verne."

# Table des matières

**1. Introduction technique** ..... **4**
**2. Architecture logicielle** ..... **4**
2.1. Modèle de Conception: Pattern MVC et Strategy ..... 4
2.1.1. Le Modèle ..... 4
2.1.2. La Vue ..... 4
2.1.3. Le Contrôleur et le Duel Manager ..... 5
2.1.4. Flux d'Execution (Game Tick) ..... 5
2.2. Organisation Modulaire et Réalisation Technique ..... 7
2.2.1. Architecture des Paquetages ..... 7
2.2.2. Conception Logicielle et Diagramme de Classes ..... 8
**3 Modélisation de l'IA** ..... **11**
3.1 Approche par Recherche Heuristique: Algorithme A* ..... 11
3.1.1. Fondement Théorique et Modèle de Référence ..... 11
3.1.2. Logique Opérationnelle ..... 11
3.2 Approche par Apprentissage par Renforcement: Q-Learning Tabulaire ..... 13
3.2.1 La Q-Table: Structure et Encodage 11-bits ..... 13
3.2.2 Dynamique d'Apprentissage et Équation de Bellman ..... 13
3.2.3 Reward Shaping (Ingénierie des Récompenses) ..... 13
3.2.3 Surveillance et Convergence (TensorBoard) ..... 14
**4. Spécification des Tests** ..... **14**
4.1. Tests Unitaires et Fonctionnels (Framework Pytest) ..... 14
4.2. Tests d'Intégration et de Concurrence ..... 14
4.3. Benchmarks et Indicateurs de Performance (KPI) ..... 14
4.4. Environnement d'Exécution et Reproductibilité ..... 15
**5. Choix techniques et justification** ..... **15**
5.1. Écosystème de développement : Python 3.12 et Pygame 2.6 ..... 15
5.2. Architecture tabulaire et Encodage 11-bits ..... 15
5.3. Rigueur structurelle: Clean Architecture et Parallélisme ..... 16
5.4. Persistance des données : Migration vers MsgPack ..... 16
**6. Conclusions** ..... **16**
6.1. Bilan Architectural ..... 16
6.2. Performance et Résultats (KPI) ..... 16
6.3. Excellence Technique ..... 17
**Bibliographie** ..... **17**

# Table des images

Figure 01: Diagramme de séquence - Cycle d'exécution d'un Game Tick.....6
Figure 02: Diagramme de Paquetages..... 8
Figure 03: Paquet Agent..... 10
Figure 04: Fonction d”évaluation heuristique..... 11
Figure 05: Fonction heuristique..... 11
Figure 06: Diagramme d'activités - Logique opérationnelle de l'algorithme A* et gestion des listes..... 12

# 1. Introduction technique

Le présent rapport documente l'architecture logicielle finalisée et les choix technologiques qui structurent le système Snake AI. Ce document expose la mise en œuvre concrète d'une application robuste, modulaire et performante, répondant aux exigences de la SAE « Réaliser une Application Informatique Complète » du parcours MIAGE.

L'enjeu central de cette réalisation a consisté à intégrer et comparer deux paradigmes de décision au sein d'un même moteur de jeu, orchestrés par un environnement de développement moderne sous Linux (WSL/Ubuntu) utilisant le gestionnaire de paquets *uv* pour garantir la reproductibilité des dépendances:

- D'une part, l'implémentation de l'algorithme A* pour la recherche de chemin optimal, s'appuyant sur une fonction heuristique de Manhattan adaptée à la grille bidimensionnelle du jeu.
- D'autre part, la mise en œuvre du Q-Learning Tabulaire, une méthode d'apprentissage par renforcement pilotée par une Q-Table. L'efficacité de cet agent repose sur un encodage d'état discret en 11 bits, permettant une convergence rapide et une prise de décision autonome après une phase d'exploration (Epsilon-greedy) rigoureusement monitorée.

L'architecture logicielle a été déployée selon le patron de conception MVC (Modèle-Vue-Contrôleur) et le Design Pattern Strategy, isolant strictement la logique métier des composants de rendu graphique (*Pygame*) et des algorithmes d'IA. Ce découpage a permis d'implémenter une suite d'outils d'automatisation via un Makefile (incluant le linting avec *Ruff* et les tests unitaires avec *Pytest*) ainsi qu'un système de suivi des performances via TensorBoard. Les sections suivantes détaillent les résultats obtenus, les spécifications algorithmiques finales et les métriques de convergence validant la fiabilité du système en mode Duel.

# 2. Architecture logicielle

## 2.1. Modèle de Conception: Pattern MVC et Strategy

L'architecture du système repose sur le patron de conception **Modèle-Vue-Contrôleur (MVC)**, plus spécifiquement dans sa variante **Passive Model** [1]. Ce choix a été validé lors de l'implémentation pour sa capacité à maintenir un flux de données unidirectionnel et centralisé, essentiel pour la synchronisation des agents d'IA.

### 2.1.1. Le Modèle

Le Modèle, centralisé dans le module 'snake_ai.engine', encapsule l'état logique de la simulation via la classe 'SnakeEngine' et la structure de données 'GameState'..

- **Réalisation**: Le modèle est resté strictement agnostique. Il valide les collisions et gère la croissance du serpent, mais ne connaît ni l'existence de l'algorithme A*, ni celle du Q-Learning. Cette neutralité a permis de réaliser des test unitaires isoles avec 'Pytest' sur la logique pure du jeu.

### 2.1.2. La Vue

Gérée par le module 'Renderer' (via Pygame), la vue est strictement passive.

- **Évolution**: Dans la phase de réalisation, la vue a été optimisée pour supporter le Mode Duel. Elle est capable de rendre simultanément deux instances du modèle (A* et RL) sur un même écran, prouvant la flexibilité de la séparation Vue/Modèle initiale.

### 2.1.3. Le Contrôleur et le Duel Manager

Le contrôleur a évolué pour devenir le pivot de l'exécution, notamment à travers l'implémentation du 'DuelManager'. Son rôle est articulé autour de deux axes:

- **Orchestration du Cycle de Vie (Game Loop)** : Il gère la boucle de jeu découplée. En phase d'entraînement, le rendu peut être désactivé pour accélérer les calculs.
- **Médiation Parallèle (Design Pattern Strategy)**: Le contrôleur utilise le pattern Strategy pour interroger les agents.
    - Pour le mode Duel, le contrôleur utilise un ThreadPoolExecutor (multithreading). Cela permet de solliciter simultanement l'agent A* (calcul de chemin) et l'agent RL (inférence Q-Table) sans que la latence de l'un ne bloque l'autre, garantissant une confrontation équitable.

Cette architecture permet une interchangeabilité totale. L'ajout d'un nouvel agent (ex: Deep Q-Learning) ne nécessiterait aucune modification du moteur de jeu ou de l'interface graphique, respectant ainsi le principe **Open/Closed** des principes SOLID.

### 2.1.4. Flux d'Execution (Game Tick)

La dynamique opérationnelle repose sur un cycle synchrone:

1. Le Contrôleur récupère l'état actuel (GameState) du Modèle.
2. Il transmet cet état à l'Agent (Stratégie) qui retourne une action.
3. Le Contrôleur injecte l'action dans le Modèle pour mettre à jour l'état physique.
4. La Vue est notifiée pour rafraîchir l'affichage graphique.

*Pour illustrer cette dynamique opérationnelle et le rôle de médiateur du contrôleur, le diagramme de séquence suivant (voir Figure 01) détaille le flux d'un cycle d'exécution (Game Tick). On y observe l'interaction synchrone entre l'agent (la stratégie), le moteur de jeu (le modèle) et la vue passive.*

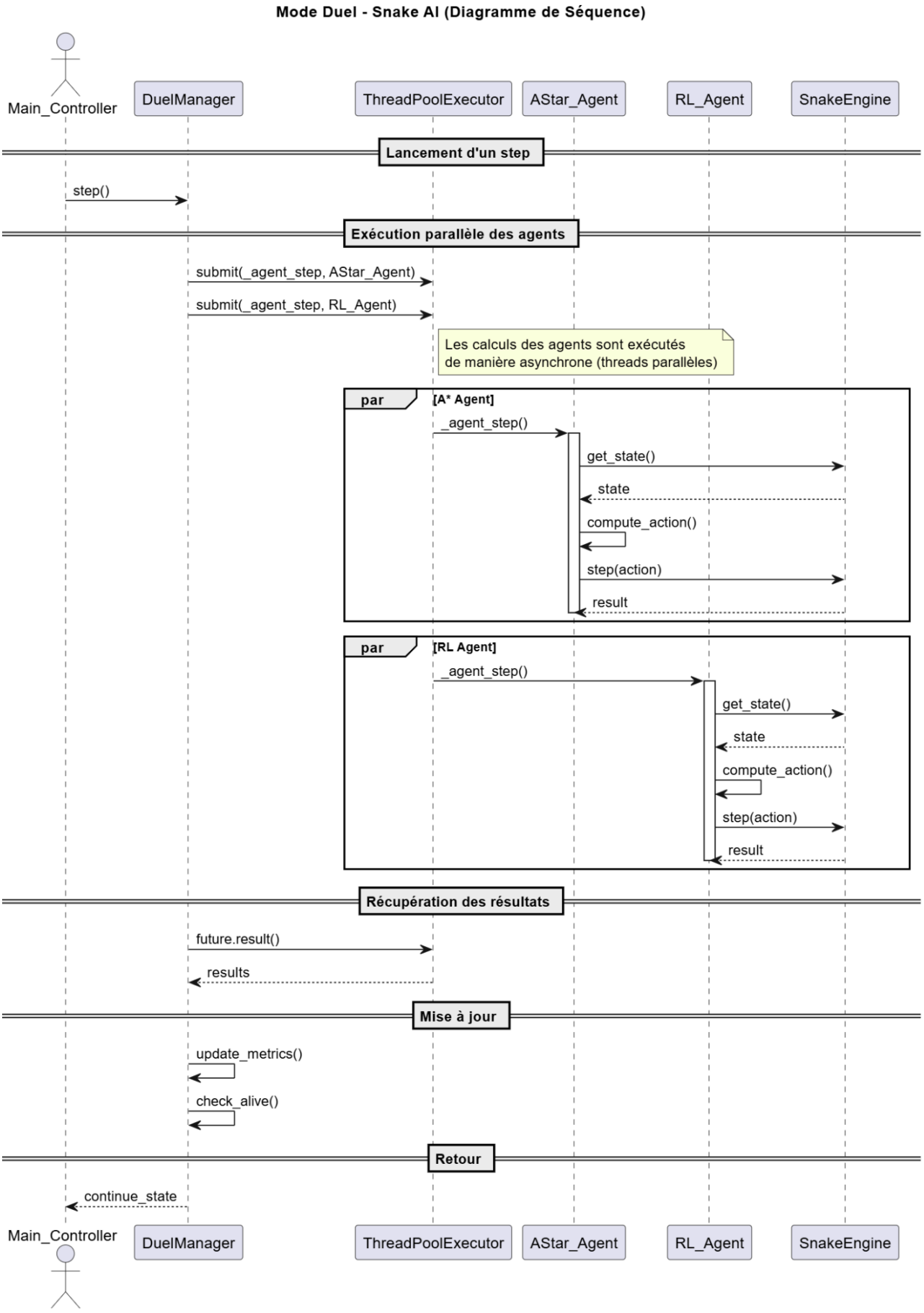



*Figure 01: Diagramme de séquence - Cycle d'exécution d'un Game Tick*

## 2.2. Organisation Modulaire et Réalisation Technique

L'architecture finale du système a été consolidée pour respecter les principes de la Clean Architecture et les principes SOLID. L'évolution majeure par rapport à la conception initiale réside dans l'unification des composants via un environnement de gestion moderne (uv) et l'ajout d'une couche d'orchestration avancée pour la comparaison des agents.

### 2.2.1. Architecture des Paquetages

Le système est désormais structuré en un package Python nommé 'snake_ai', garantissant une modularité totale et une facilité d'installation:

- 'snake_ai.engine' (Modèle): Contient la logique pure de jeu (serpent, nourriture, collisions). Ce module est resté agnostique de toute IA ou interface graphique.
- 'snake_ai.agents' (Stratégie): 'agents.rl' contient le 'QTableAgent' et le 'Trainer' (pour l'apprentissage) et 'agents.astar' implémente la recherche de chemin heuristique.
- 'snake_ai.gui' (Vue): Gère le rendu via Pygame, désormais optimisé pour l'affichage multi-instances (Mode Duel).
- 'snake_ai.utils' (Analytics & IO): Regroupe la persistance (MsgPack) et le 'MetricsCollector' pour tensorboard.

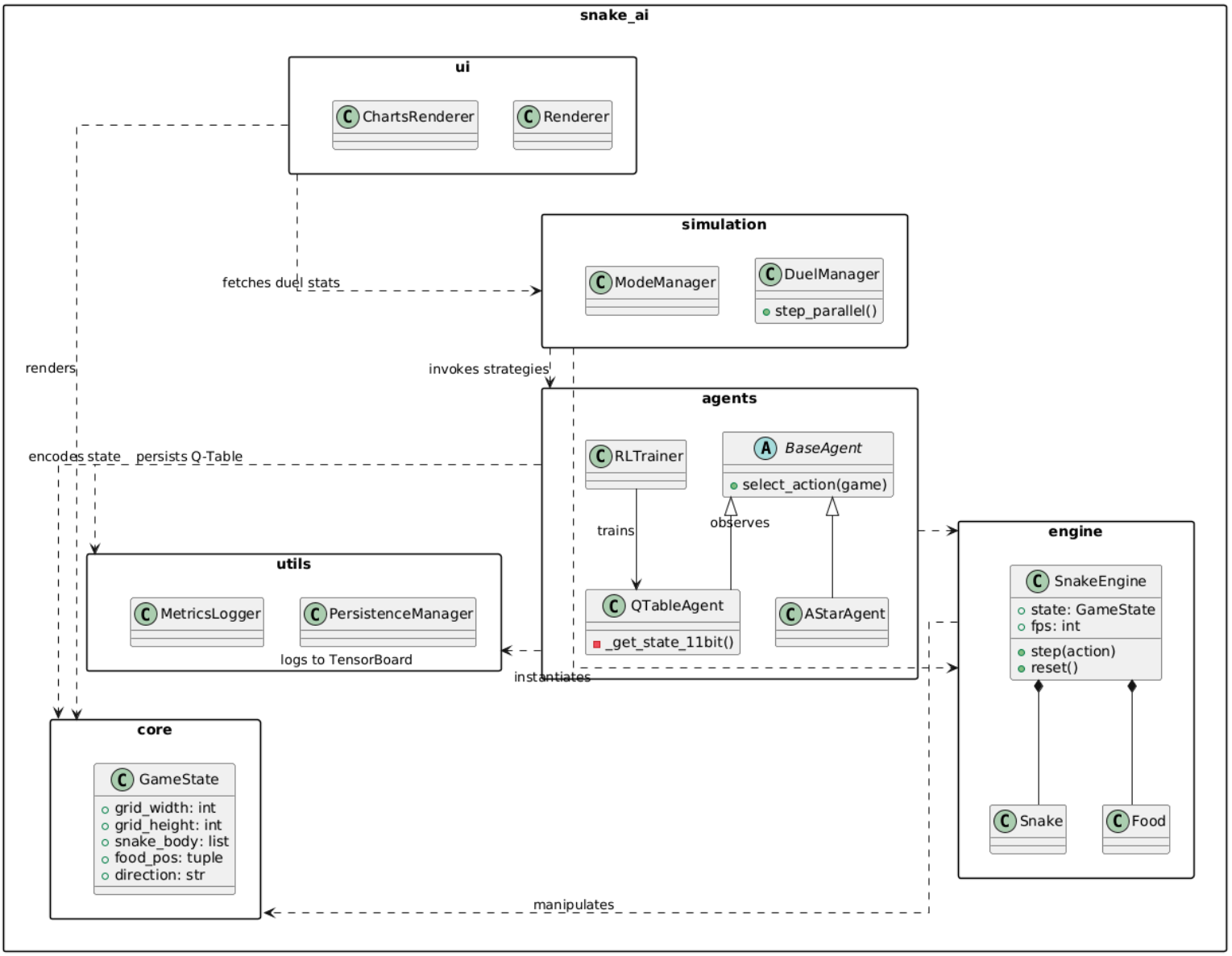



*Figure 02: Diagramme de Paquetages*

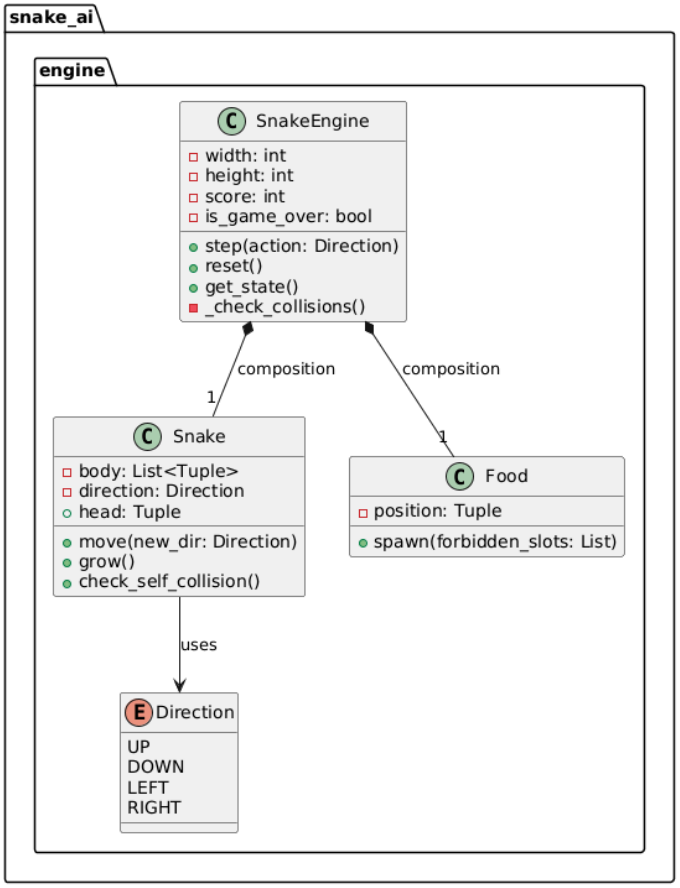
### 2.2.2. Conception Logicielle et Diagramme de Classes

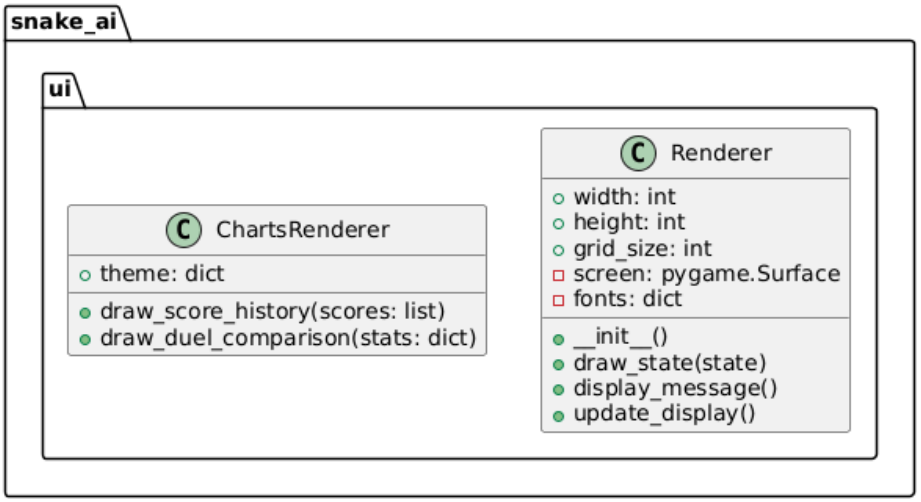
L'architecture du projet repose sur les principes SOLID, garantissant la modularité et l'extensibilité du système.

#### A. La noyau de Simulation (Package 'engine' & 'core')

Le moteur de jeu applique une **composition forte** pour garantir l'intégrité du cycle de vie des entités:

- **SnakeEngine**: Pilote la logique. Il détient une instance de **GameState** (données), de **Snake** et de **Food**.
- **GameState**: Centralisé l'état physique (positions, score, direction) pour une manipulation thread-safe.
- **Direction (Enum)**: Normalise les vecteurs de mouvement, éliminant les constantes dispersées.




## B. L'intelligence Artificielle (Package Agent)

La modularité repose sur le Pattern Strategy et le polymorphisme :

- **BaseAgent (ABC):** Classe abstraite définissant le contrat select_action(state).
- **AStarAgent & QTableAgent**: Implémentations concrètes. Cette structure permet d'interchanger les algorithmes (Heuristique vs Apprentissage) sans modifier le moteur.
- **Découplage**: L'agent est agnostique de l'interface graphique ; il ne traite que l'encodage logique (ex: l'état 11-bits).

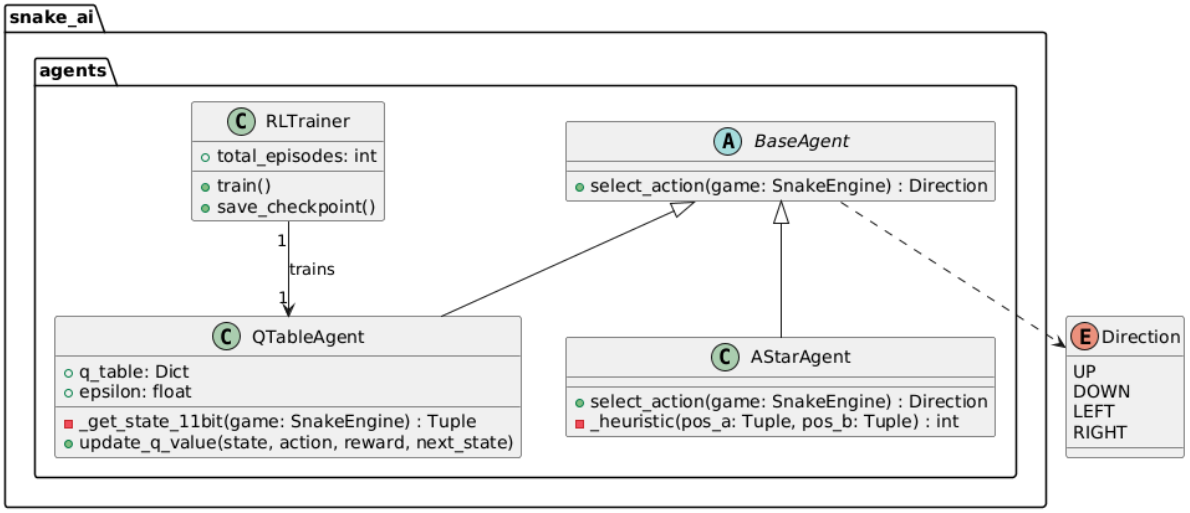
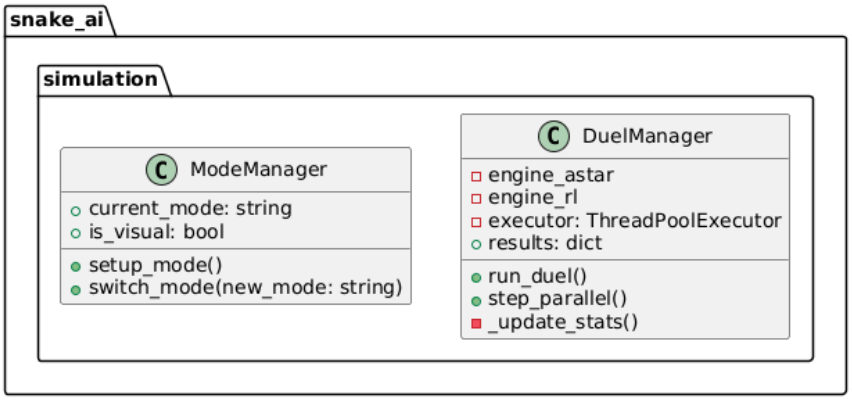
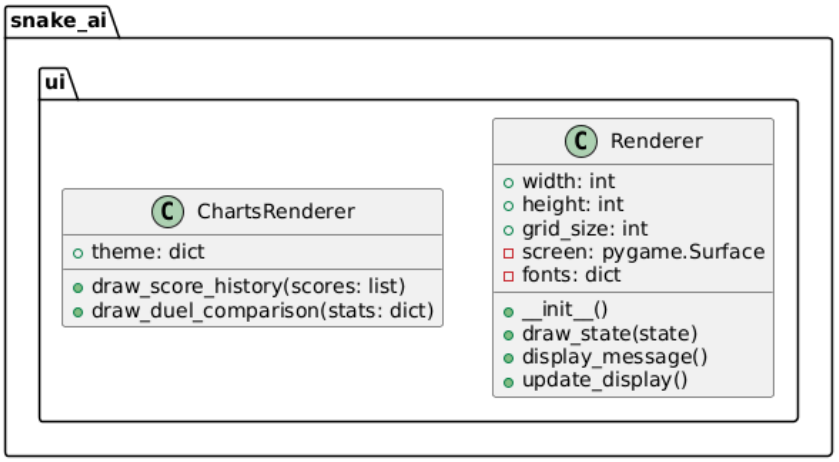



*Figure 03: Paquet Agent*

### C. L'Orchestration et la Vue (Package simulation & ui)

- **DuelManager:** Classe pivot utilisant le multithreading (ThreadPoolExecutor) pour orchestrer deux instances de jeu simultanées.
- **Renderer :** Vue passive utilisant Pygame. Elle reçoit le GameState pour le rendu, assurant une séparation stricte entre logique métier et interface.




### D. Services Transverses

- **PersistenceManager** : Gère les flux I/O (MsgPack/JSON). Il délègue la sauvegarde de la Q-Table, respectant le principe de Responsabilité Unique (SRP).
- **MetricsLogger** : Extrait les indicateurs clés (KPI) vers TensorBoard, permettant une analyse comparative rigoureuse de la convergence du Q-Learning par rapport à l'A*.

# 3 Modélisation de l'IA

## 3.1 Approche par Recherche Heuristique: Algorithme A*

### 3.1.1. Fondement Théorique et Modèle de Référence

Le moteur de recherche suit le formalisme de Russell et Norvig [2] en évaluant chaque nœud $n$ via la fonction de coût combinée :

$$f(n) = g(n) + h(n)$$

*Figure 04: Fonction d"évaluation heuristique*

- **Coût réel g(n)** : Représente la distance parcourue depuis la tête du serpent (coût unitaire $c=1$ par cellule).
- **Heuristique h(n)** : Pour garantir l'optimalité, nous utilisons la Distance de Manhattan, car les mouvements sont strictement orthogonaux (4-connected graph) :

$$h(n) = |x_{target} - x_n| + |y_{target} - y_n|$$

*Figure 05: Fonction heuristique*

### 3.1.2. Logique Opérationnelle

L'algorithme gère dynamiquement deux listes pour explorer l'espace d'états :

1. **Open List** : Une file de priorité stockant les nœuds à explorer, triés par $f(n)$ croissant.
2. **Closed List** : Stocke les nœuds déjà évalués pour éviter les cycles et les recalculs inutiles.

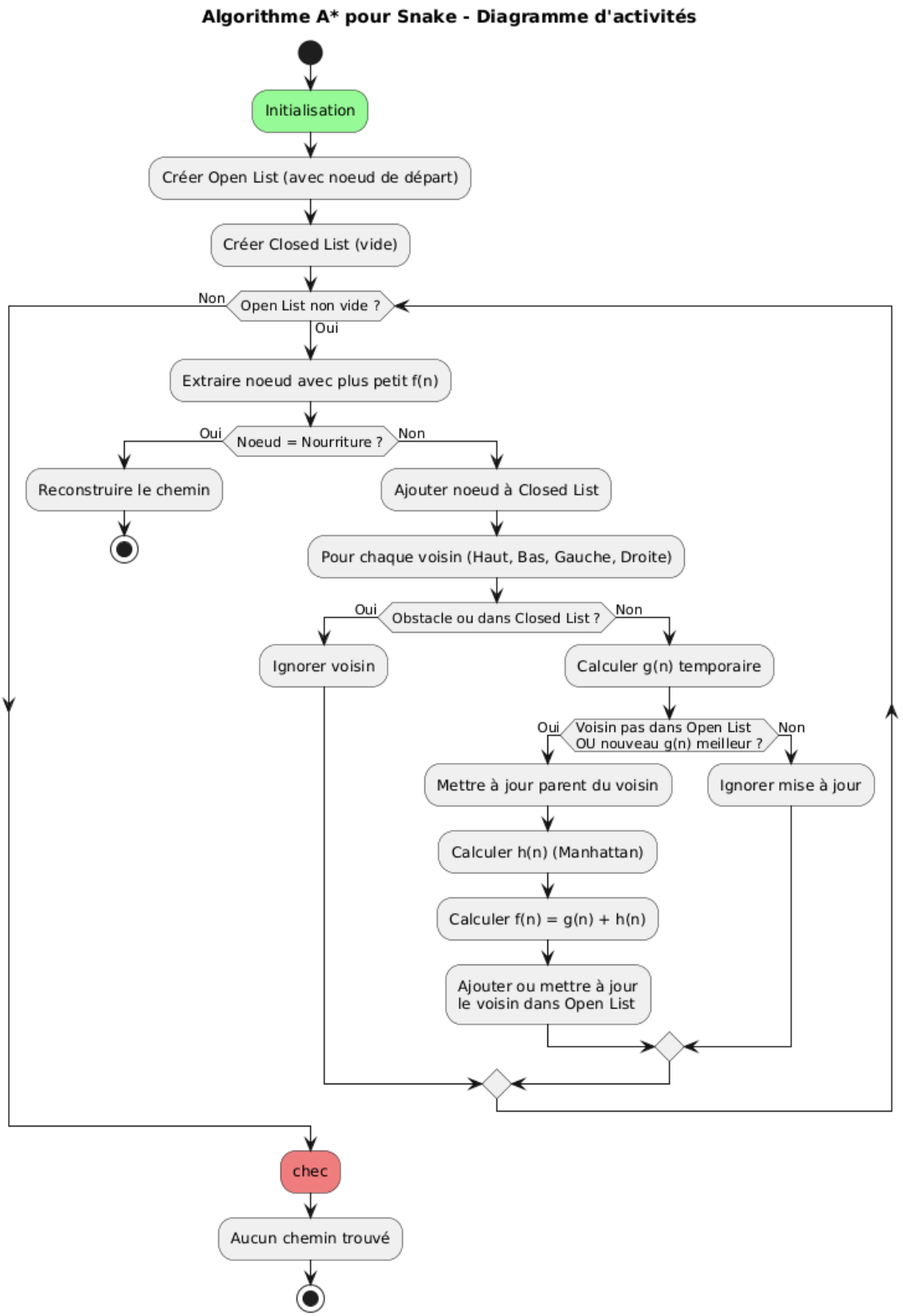



*Figure 06: Diagramme d'activités - Logique opérationnelle de l'algorithme A* et gestion des listes*

## 3.2 Approche par Apprentissage par Renforcement: Q-Learning Tabulaire

Le Q-Learning est une méthode d'apprentissage “sans modèle" où l'intelligence de l'agent est stockée dans une Q-Table, une structure de données agissant comme une mémoire associative.

### 3.2.1 La Q-Table: Structure et Encodage 11-bits

La Q-Table est un dictionnaire où chaque clé représente un état unique du jeu et chaque valeur un vecteur d'actions possibles.

- **L’Espace d’États (L'Indice 11-bits)** : Pour éviter l'explosion combinatoire, nous utilisons un encodage binaire compact de 11 bits (2048 états possibles) :
    - **Danger (3 bits)** : Proximité immédiate d'un obstacle (Devant, Gauche, Droite).
    - **Direction Actuelle (4 bits)** : Cap actuel du serpent (N, S, E, O).
    - **Position Nourriture (4 bits)** : Emplacement relatif de la cible (Nord, Sud, Est, Ouest).
- **Les Actions (Les Colonnes)** : L'agent choisit entre trois actions relatives : {Continuer, Tourner à Gauche, Tourner à Droite}.
- **Persistance via MsgPack** : La table est sérialisée en format binaire MsgPack. Ce choix technique garantit une vitesse de lecture/écriture nettement supérieure au JSON, cruciale pour charger instantanément des milliers d'états lors du lancement.

### 3.2.2 Dynamique d’Apprentissage et Équation de Bellman

L'apprentissage repose sur un cycle itératif d'interaction avec le SnakeEngine :

1. **Exploration vs Exploitation ($\epsilon$-greedy)** :
   Au départ, *epsilon* = 1.0$ (actions 100% aléatoires). Au fil des épisodes, *epsilon* décroît (decay), forçant l'agent à exploiter les valeurs optimales déjà stockées dans la Q-Table.
2. **Mise à jour de la Valeur Q** :
   À chaque mouvement, la "connaissance" est ajustée via l'équation de Bellman simplifiée :

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max Q(s',a') - Q(s,a)]$$

Où *alpha* est le taux d'apprentissage et *gamma* le facteur de remise (importance du futur).

### 3.2.3 Reward Shaping (Ingénierie des Récompenses)

Le comportement de l'IA est dicté par une fonction de récompense asymétrique :

- **Manger (+20)** : Objectif principal.

- **Collision (-10)** : Sanction critique mettant fin à l'épisode.
- **Rapprochement (+1.0) / Éloignement (-1.5)** : Encourage une navigation directe vers la nourriture et pénalise les boucles infinies.

### 3.2.3 Surveillance et Convergence (TensorBoard)

La validité de l'apprentissage est vérifiée par deux indicateurs dans *MetricsLogger*:

- **Taille de la Q-Table** : Le nombre d'états découverts. Une stabilisation de la courbe indique que l'agent a exploré la majorité des situations critiques.
- **Évolution du Score Moyen** : Confirme que l'agent maximise effectivement ses gains cumulés au fil des milliers d'épisodes d'entraînement.

# 4. Spécification des Tests

## 4.1. Tests Unitaires et Fonctionnels (Framework Pytest)

L'automatisation via *pytest* permet d'isoler et de valider chaque module critique :

- **Moteur de Jeu** (*engine*) : Validation des collisions (murs et corps) et de la cohérence du GameState après chaque step().
- **Encodage d'État** (*agents.rl.encoders*) : Vérification cruciale que la transformation des observations en vecteur 11-bits est déterministe et sans erreur.
- **Persistance** (*utils.persistence*) : Test d'intégrité de la sérialisation MsgPack. On vérifie que la Q-Table chargée est identique à celle sauvegardée (comparaison de hash).

## 4.2. Tests d'Intégration et de Concurrence

Ces tests valident la communication entre les packages selon l'architecture découplée du projet :

- **Flux Simulation-Agent** : Vérification que le *DuelManager* transmet correctement l'état aux agents et reçoit une *Direction* valide en retour.
- **Robustesse Multithread** : Test de non-régression sur l'exécution parallèle des deux moteurs de jeu dans le *DuelManager* pour s'assurer qu'il n'y a pas de "Race Condition" sur l'accès aux ressources.
- **Synchronisation UI** : Validation du rafraîchissement du *Renderer* par rapport aux changements d'état du moteur (Fréquence cible : 10 FPS minimum).

## 4.3. Benchmarks et Indicateurs de Performance (KPI)

L’évaluation finale de l’intelligence artificielle a été réalisée via un protocole de 50 duels simultanés sur une grille de 20x20. Ce benchmark compare l'efficacité de l'algorithme déterministe (A*) face à l'apprentissage par renforcement (RL) après convergence du modèle.

| Indicateur (KPI) | Objectif A* (Heuristique) | Objectif Q-Learning (RL) |
|---|---|---|
| **Score Moyen** | 22.54 (±13.36) | 11.24 (±6.58) |
| **Pas Moyens (Steps)** | 312.6 | 311.8 |
| **Connaissances (États)** | N/A | 1258 états connus |
| **Fiabilité** | Optimalité théorique | Décisions stables et rapides |

## 4.4. Environnement d'Exécution et Reproductibilité

Pour garantir la stabilité des mesures, les tests sont exécutés dans l'environnement suivant :

- **Stack Technique** : Python 3.12, Pygame 2.5+, Gestionnaire de paquets uv.
- **Système** : Linux (Ubuntu/WSL2) pour une gestion optimale des processus.
- **Mode "Headless"** : Les sessions d'entraînement de 20 000 épisodes sont réalisées sans rendu graphique pour maximiser la vitesse de calcul et la collecte de logs via **TensorBoard**.

# 5. Choix techniques et justification

## 5.1. Écosystème de développement : Python 3.12 et Pygame 2.6

Le choix de **Python 3.12** s'est imposé pour sa gestion optimisée de la mémoire et sa prédominance en IA.

- **Gestionnaire de paquets** uv : Contrairement au pip classique, nous utilisons uv pour une installation de dépendances ultra-rapide et une gestion déterministe de l'environnement via uv.lock.
- **Automatisation via Makefile** : L'utilisation d'un Makefile permet d'abstraire la complexité des commandes (make train, make benchmark, make test), offrant une expérience développeur (DX) fluide et standardisée.
- **Bibliothèque Pygame** : Nous avons maintenu Pygame pour sa légèreté. Elle nous permet de contrôler finement la **Game Loop** sans la surcharge d'un moteur de jeu complet, facilitant l'exécution des agents en mode "headless" lors des entraînements intensifs.

## 5.2. Architecture tabulaire et Encodage 11-bits

Plutôt que d'utiliser des réseaux de neurones complexes (Deep RL), nous avons opté pour le **Q-Learning Tabulaire**.

- **Optimisation de l'Espace d'États** : L'innovation majeure réside dans l'encodage 11-bits. En compressant l'environnement (danger, direction, nourriture) dans ce

format, nous évitons "l'explosion combinatoire" tout en conservant une précision chirurgicale.
- **Efficacité et Portabilité** : Cette approche garantit une convergence rapide (quelques minutes d'entraînement suffisent) et une exécution en temps réel sans nécessiter de GPU, rendant le système extrêmement portable.

## 5.3. Rigueur structurelle: Clean Architecture et Parallélisme

L'adoption d'une architecture découpée en packages (core, engine, agents, simulation, ui, utils) respecte les principes SOLID.

- **Multithreading (DuelManager)** : Pour les benchmarks, nous avons implémenté un système de threads via ThreadPoolExecutor. Cela permet de faire tourner deux instances du moteur en parallèle, garantissant que les comparaisons de score entre A* et RL sont réalisées dans des conditions temporelles identiques.
- **Séparation stricte (MVC)** : Le moteur (engine) ne connaît pas l'interface (ui). Cette isolation permet de tester la logique pure du jeu via Pytest indépendamment du rendu graphique.

## 5.4. Persistance des données : Migration vers MsgPack

Initialement prévu en JSON, nous avons migré la persistance de la Q-Table vers le format **MsgPack**.

- **Performance I/O** : MsgPack est un format binaire compressé beaucoup plus rapide à sérialiser/désérialiser que le JSON.
- **Volume de données** : Pour des modèles contenant plus de 1200 états, le format binaire réduit considérablement la taille sur disque tout en évitant les erreurs de parsing liées aux types de données Python complexes.

# 6. Conclusions

## 6.1. Bilan Architectural

- **Découplage MVC** : La séparation entre le moteur (engine) et l'interface (ui) a permis une maintenance isolée et des tests unitaires fiables.

- **Flexibilité IA** : Le **Pattern Strategy** (BaseAgent) a assuré l'interchangeabilité immédiate entre A* et RL sans modifier le code source de la simulation.

## 6.2. Performance et Résultats (KPI)

- **Efficacité A*** : Optimalité confirmée avec un score moyen de **22.54**. C'est le référentiel de survie déterministe.

- **Apprentissage RL** : Succès de l'encodage **11-bits** (réduction drastique de l'espace d'états). L'agent a stabilisé sa politique sur **1258 états connus** (score moyen : **11.24**).

- **Vitesse** : Temps de décision en $O(1)$ pour le RL (accès direct à la Q-Table), surpassant la réactivité de l'A* en situation de jeu réelle.

## 6.3. Excellence Technique

- **Persistance** : Migration réussie vers **MsgPack**, offrant une rapidité I/O et une compacité supérieures au JSON.
- **Concurrence** : Utilisation du multithreading (ThreadPoolExecutor) pour des benchmarks massifs et impartiaux.
- **Environnement** : Standardisation du workflow via **uv** (gestion de paquets) et **Makefile**.

# Bibliographie


**[1] GAMMA, Erich, et al.** *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

**[2] Russell, S., & Norvig, P**. (2020). *Artificial Intelligence: A Modern Approach*. Pearson.